

Guía Definitiva: Ecosistema de Desarrollo Autónomo con IA

Este documento define la arquitectura, herramientas y flujos de trabajo operativos para un entorno de desarrollo asistido por IA basado en `claude CLI`, optimizado para máxima precisión y mínimo consumo de tokens, manteniendo a `Claude 3 Opus` como motor de planificación estratégica.

1. Arquitectura del Stack Base

Tu entorno se compone de 5 pilares fundamentales. Cada uno tiene una responsabilidad única y aislada:

Componente	Herramienta	Rol en el Ecosistema	Cuándo actúa
CLI / Motor Base	<code>claude CLI</code>	El ejecutor nativo. Habla con la API de Anthropic y gestiona las herramientas locales.	Siempre activo.
Memoria	Engram	Persistencia de contexto inter-sesiones. Evita repetir reglas arquitectónicas a la IA.	Siempre activo.
Orquestador Ligero	Gentle (SDD)	Tareas acotadas, refactoros locales y features de tamaño medio.	Bajo demanda.
Orquestador Pesado	GSD	Épicas, migraciones masivas y diseño de nuevas arquitecturas. Utiliza Opus para el <i>planning</i> .	Bajo demanda.
Filtro Terminal	RTK	Intercepta logs kilométricos de consola (tests, builds) y los resume.	Pasivo (background).

2. Las 3 Capas de Optimización de Tokens

El mayor riesgo de usar Opus en GSD es la fuga masiva de tokens durante el escaneo del proyecto ("fuerza bruta" mediante `grep`). Para evitarlo, aplicamos este modelo de 3 capas:

Capa 1: Filtro de Salida (RTK)

- **Qué es:** Rust Token Killer. Un proxy transparente en tu shell.
- **Qué hace:** Comprime la salida de comandos (`git status` , `pytest` , `npm run build`). Reduce entre un 60-90% la basura que llega a la ventana de contexto de la IA.

Capa 2: Navegación Estructurada (LSP nativo de Claude)

- **Qué es:** Language Server Protocol (activo vía flag oculto en Claude Code).
- **Qué hace:** Reemplaza la lectura ciega de archivos y el uso ineficiente de `grep` . Da inteligencia IDE al agente: referencias exactas, saltos a definiciones y diagnósticos en

tiempo real (50ms por query frente a 30s-60s con grep).

Capa 3: Indexación y Visión Espacial (Servidores MCP)

- **Qué es:** Herramientas externas (jCodeMunch, RepoMapper) que extraen el "esqueleto" AST (Abstract Syntax Tree) del código base.
- **Qué hace:** Le da a GSD un mapa resumido o fragmentos milimétricos de clases/funciones por ID, en lugar de obligar al agente a leer módulos enteros para entender cómo se relacionan las dependencias.

3. Plan de Implementación (Setup Técnico)

Sigue este orden estricto para no saturar tu entorno.

Fase 1: Activar el LSP (Capa 2) para Python en Claude Code

Esto elimina el 80% del gasto innecesario en búsquedas.

1. **Habilitar flag experimental:** Edita (o crea) el archivo `~/.claude/settings.json` (en Windows: `C:\Users\tu_usuario\.claude\settings.json`) y añade:

```
{
  "env": {
    "ENABLE_LSP_TOOL": "1"
  }
}
```

2. **Instalar el binario del LSP para Python (Pyright):**

```
npm install -g pyright
```

3. **Activar el plugin en Claude:**

```
claude plugin marketplace update claude-plugins-official
claude plugin install pyright-lsp Python
claude plugin enable pyright-lsp
```

(Nota: Requiere reiniciar Claude Code para que el LSP empiece a indexar).

Fase 2: Medición y Evaluación

- Ejecuta GSD con el *planning* de Opus usando **solamente Capa 1 (RTK) y Capa 2 (LSP)**.
- Monitoriza el gasto con el comando `/cost` en Claude.
- **Regla de oro:** Si el consumo de tokens baja a niveles financieros aceptables, detén la instalación aquí. La simplicidad del stack previene alucinaciones.

Fase 3: Contingencia (Capa 3 MCP)

Si, y solo si, GSD (Opus) sigue quemando demasiados tokens intentando mapear repositorios monolíticos gigantes, elige UNA de estas herramientas e inyéctala vía MCP (`claude mcp add ...`):

- **Opción A (RepoMapper):** Usa PageRank para clasificar símbolos por importancia. Ideal para orientación direccional ("¿Qué archivos importan realmente en este repo enredado?").
- **Opción B (jCodeMunch):** Recuperación basada en Tree-sitter. Ideal para extracción quirúrgica precisa ("Saca solo los 50 bytes que conforman esta clase").

4. Herramientas Descartadas (Lista Negra Temporal)

Para evitar redundancias de red y conflictos cognitivos en el agente, NO instales las siguientes herramientas:

- **✗ Mapa de Repositorio Aider:** Es nativo y exclusivo del CLI de Aider. Incompatible con el `stack claude CLI`.
- **✗ Scope:** Fase alfa temprana (Solo soporta TS y C#). Basado en dependencias SQLite que añaden fricción innecesaria.
- **✗ Serena:** Altamente maduro, pero su set de herramientas (`find_symbol` , referencias cruzadas) se solapa en un 90% con el LSP nativo que acabamos de activar en la Fase 1. Redundante.

5. Árbol de Decisión: ¿Cuándo usar qué?

Antes de lanzar un prompt, aplica esta lógica de enrutamiento:

Escenario A: Mantenimiento, Refactores y Features Medianas

- **Orquestador:** Gentle (SDD).
- **Modelo de IA:** Sonnet 3.7.
- **Flujo:**
 1. Engram carga el estado y directrices.
 2. Gentle formula la solución técnica.
 3. El agente usa LSP (Capa 2) para navegar y modificar directamente las definiciones requeridas.
 4. RTK (Capa 1) silencia el ruido de los tests unitarios ejecutados.
- **Uso de Capa 3:** NO. Prohibido por sobreingeniería.

Escenario B: Epicas, Diseño Arquitectónico y Código Desconocido

- **Orquestador:** GSD (Get Shit Done).
- **Modelo de IA:** Opus (Solo para Planning) -> Sonnet (Para ejecución).
- **Flujo:**

1. GSD + Opus inician la planificación.
2. *[Opcional]* Si el repositorio es gigante, GSD consulta el servidor MCP de la Capa 3 (RepoMapper/jCodeMunch) para obtener un grafo de dependencias de bajo costo.
3. Opus entrega el plan de ejecución segmentado.
4. Se delega la ejecución al agente (Sonnet) usando LSP (Capa 2) para escribir el código con precisión.
5. RTK (Capa 1) captura cualquier crash mayor en los tests de integración y retroalimenta al agente.